

- Description

This module manages flight bookings and seat allocation, ensuring that:

1. **Seats are assigned based on availability**—no overbooking allowed.
2. **Passengers can choose economy or business class**, with different seat limits.
3. **A waiting list system** is implemented for passengers if a flight is fully booked.
4. **Cancellations free up seats** and notify the first waiting list passenger if applicable.
5. **A booking history log** is maintained to track reservations and cancellations.

This system ensures efficient seat allocation while preventing overbooking.

- Specification

module FlightBookingSystem

var economySeats, businessSeats: integer

var waitingList: sequence of string

var bookingHistory: sequence of string

process bookSeat(passenger: string, isBusinessClass: boolean) flag: boolean

functional scenario form:

isBusinessClass > 0 and businessSeats > 0 && flag = true and businessSeats = businessSeats - 1 and bookingHistory = bookingHistory $\oplus$ ("Business Class Booking Confirmed"), Valid booking, assigned to Business Class

||

¬isBusinessClass > 0 and economySeats > 0 && flag = true and economySeats = economySeats - 1 and bookingHistory = bookingHistory $\oplus$ ("Economy Class Booking Confirmed"), Valid booking, assigned to Economy Class

||

businessSeats = 0 and economySeats = 0 && flag = false and waitingList = waitingList $\oplus$ (passenger), Flight full, passenger added to waiting list

end process

process cancelBooking(passenger: string, isBusinessClass: boolean) flag: boolean

functional scenario form:

(passenger $\in$ bookingHistory) && flag = true $\wedge$ businessSeats = businessSeats + 1 $\wedge$ bookingHistory = bookingHistory $\oplus$ ("Business Class Booking Cancelled") $\wedge$ (if |waitingList| > 0 then bookSeat(waitingList[0], true)), Booking cancelled, seat available, waiting list updated

OR

(passenger $\notin$ bookingHistory) && flag = true $\wedge$ economySeats = economySeats + 1 $\wedge$ bookingHistory = bookingHistory $\oplus$ ("Economy Class Booking Cancelled") $\wedge$ (if |waitingList| > 0 then bookSeat(waitingList[0], false)), Invalid cancellation, no action

end process

end module

- Java Code

```
package code;
```

```

import java.util.*;
class Flight {
    private String flightNumber;
    private int economySeats;
    private int businessSeats;
    private Queue<String> waitingList;
    private List<String> bookingHistory;
    public Flight(String flightNumber, int economySeats, int businessSe
ats) {
        this.flightNumber = flightNumber;
        this.economySeats = economySeats;
        this.businessSeats = businessSeats;
        this.waitingList = new LinkedList<>();
        this.bookingHistory = new ArrayList<>();
    }
    public boolean bookSeat(String passenger, boolean isBusinessClass)
{
    if (isBusinessClass) {
        if (businessSeats > 0) {
            businessSeats--;
            bookingHistory.add(String.format("Booking: %s in Busine
ss Class, Remaining: %d", passenger, businessSeats));
            System.out.printf("Booking successful! %s assigned to B
usiness Class. Remaining: %d\n", passenger, businessSeats);
            return true;
        }
    } else {
        if (economySeats > 0) {
            economySeats--;
            bookingHistory.add(String.format("Booking: %s in Econom
y Class, Remaining: %d", passenger, economySeats));
            System.out.printf("Booking successful! %s assigned to E
conomy Class. Remaining: %d\n", passenger, economySeats);
            return true;
        }
    }
    waitingList.add(passenger);
    System.out.printf("Flight full! %s added to the waiting list.%n
", passenger);
    return false;
}
    public boolean cancelBooking(String passenger, boolean isBusinessCl
ass) {
        if (isBusinessClass) {

```

```

        businessSeats++;
    } else {
        economySeats++;
    }
    bookingHistory.add(String.format("Cancellation: %s, Seat released", passenger));
    System.out.printf("Booking canceled: %s. Seat is now available.%n", passenger);
    if (!waitingList.isEmpty()) {
        String nextPassenger = waitingList.poll();
        bookSeat(nextPassenger, isBusinessClass);
        System.out.printf("Notifying waiting list passenger: %s%n", nextPassenger);
    }
    return true;
}

public void printBookingHistory() {
    System.out.println("Booking History for Flight " + flightNumber + ":");
    for (String log : bookingHistory) {
        System.out.println(log);
    }
}
}

public class c08_FlightBookingSystem {
    public static void main(String[] args) {

        // testcase-VT:
        // Flight flight = new Flight("HU789", 2, 1);
        // flight.bookSeat("Alice", false);
        // flight.bookSeat("Bob", false);
        // flight.bookSeat("Charlie", false);
        // flight.bookSeat("David", true);
        // flight.bookSeat("Eve", true);
        // flight.cancelBooking("Bob", false);
        // flight.cancelBooking("David", true);
        // flight.cancelBooking("Alice", true);
        // flight.printBookingHistory();

        // testcase-FT:
        // Flight flight1 = new Flight("HU123", 2, 2);
        // flight1.bookSeat("z?p", false);
        // flight1.bookSeat("z?", false);
        // flight1.bookSeat("UU", false);
    }
}

```

```

//      flight1.bookSeat("???-w", true);
//      Flight flight2 = new Flight("HU123", 2, 0);
//      flight2.bookSeat("???-s", true);
//      flight2.bookSeat("sad?", true);
//      flight2.bookSeat("sADD", false);
//      flight2.bookSeat("?ad---s", true);

// testcase-Z3:
//      Flight flight = new Flight("HU789", 2, -1);
//      flight.bookSeat("Alice", false);
//      flight.bookSeat("Bob", false);
//      flight.bookSeat("Charlie", false);
//      flight.bookSeat("David", true);
//      flight.bookSeat("Eve", true);
//      Flight flight2 = new Flight("HU123", 2, 0);
//      flight.bookSeat("David", true);
//      flight.bookSeat("Eve", true);
//      flight.bookSeat("David", true);
//      flight.printBookingHistory();

// testcase-UVT:
Flight flight = new Flight("HU789", 2, -1);
    flight.bookSeat("Alice", false);
    flight.bookSeat("Bob", false);
    flight.bookSeat("Charlie", false);
    flight.bookSeat("David", true);
    flight.bookSeat("Eve", true);
    Flight flight2 = new Flight("HU123", 2, 0);
    flight2.bookSeat("David", true);
    flight2.bookSeat("Eve", true);
    flight.cancelBooking("David", true);
    flight.printBookingHistory();
}
}

```

- Coverage report

1. Test case number: 8
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.5 %	12	806	818
▼ src	1.5 %	12	806	818
▼ code	1.5 %	12	806	818
▼ c08_FlightBookingSystem.java	100.0 %	12	0	12
> c08_FlightBookingSystem		0	0	0
> Flight	100.0 %	12	0	12

3. Vibration testing instruction coverage: 98.9%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.7 %	265	15,273	15,538
▼ src	1.7 %	265	15,273	15,538
▼ code	1.7 %	265	15,273	15,538
▼ c08_FlightBookingSystem.java	98.9 %	265	3	268
> c08_FlightBookingSystem	94.3 %	50	3	53
> Flight	100.0 %	215	0	215

4. Fuzz testing branch coverage: 50%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.7 %	6	812	818
▼ src	0.7 %	6	812	818
▼ code	0.7 %	6	812	818
▼ c08_FlightBookingSystem.java	50.0 %	6	6	12
> Flight	50.0 %	6	6	12
> c08_FlightBookingSystem		0	0	0

5. Fuzz testing instruction coverage: 68.1%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.2 %	186	15,357	15,543
▼ src	1.2 %	186	15,357	15,543
▼ code	1.2 %	186	15,357	15,543
▼ c08_FlightBookingSystem.java	68.1 %	186	87	273
> Flight	60.9 %	131	84	215
> c08_FlightBookingSystem	94.8 %	55	3	58

6. Z3 branch coverage: 58.3%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.8 %	7	833	840
▼ src	0.8 %	7	833	840
▼ code	0.8 %	7	833	840
▼ c08_FlightBookingSystem.java	58.3 %	7	5	12
> Flight	58.3 %	7	5	12
> c08_FlightBookingSystem		0	0	0

7. Z3 instruction coverage: 60.7%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	0.8 %	167	21,926	22,093
▼ src	0.8 %	167	21,926	22,093
▼ code	0.8 %	167	21,926	22,093
▼ c08_FlightBookingSystem.java	60.7 %	167	108	275
> Flight	51.2 %	110	105	215
> c08_FlightBookingSystem	95.0 %	57	3	60

8. Unimproved V-method branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	1.1 %	9	831	840
▼ src	1.1 %	9	831	840
▼ code	1.1 %	9	831	840
▼ c08_FlightBookingSystem.java	75.0 %	9	3	12
> Flight	75.0 %	9	3	12
> c08_FlightBookingSystem		0	0	0

9. Unimproved V-method instruction coverage: 81.5%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.0 %	224	21,869	22,093
▼ src	1.0 %	224	21,869	22,093
▼ code	1.0 %	224	21,869	22,093
▼ c08_FlightBookingSystem.java	81.5 %	224	51	275
> Flight	77.7 %	167	48	215
> c08_FlightBookingSystem	95.0 %	57	3	60